Singapore School Kinderland

Secondary 1 (Sec 1) Computer Science — Term 2 Comprehensive Study Material

Prescribed Textbook: Cambridge IGCSE & O Level Computer Science (Second Edition) | Chapter 2: Data Transmission (Pages 54–74)

PART 1: CHAPTER 2.2 — METHODS OF ERROR DETECTION (TB Pages 54–62)

During data transmission across communication networks, electrical interference, power surges, signal attenuation, or packet collisions can cause binary bits to flip ($0 \rightarrow 1$ or $1 \rightarrow 0$). Computer scientists use mathematical error detection methods to identify corrupted packets.

1. Parity Checks (Odd & Even Parity) — TB Pages 54–55

A byte is made of 8 bits: 7 bits of actual data and 1 **Parity Bit** allocated to ensure the total number of 1s in the byte conforms to an established parity rule.

- **Even Parity:** The parity bit is set to 0 or 1 so that the total count of 1s in the 8-bit byte is an **EVEN** number (0, 2, 4, 6, 8).
- **Odd Parity:** The parity bit is set so that the total count of 1s in the byte is an **ODD** number (1, 3, 5, 7).

```
Example (Even Parity): Data = 1 0 1 1 0 0 1 (Count of 1s = 4, already even) →
Parity Bit = 0 → Byte: 0 1 0 1 1 0 0 1
Example (Odd Parity): Data = 1 0 1 1 0 0 1 (Count of 1s = 4, currently even) →
Parity Bit = 1 → Byte: 1 1 0 1 1 0 0 1
```

> **Cambridge Exam Limitation:** A simple parity check cannot detect errors if an **even number of bits** flip in the same byte (e.g. two 0s flip to 1s). The total count of 1s remains even/odd, causing the corruption to go completely undetected!

2. Parity Blocks / Parity Byte Matrix — TB Pages 56–57

To overcome single-byte limitations, data bytes are assembled into a 2D block (matrix). Horizontal parity bits are appended to every row, and an extra vertical **Parity Byte** is added at the bottom.

When transmitted, if a bit flips, both its horizontal row parity and its vertical column parity will show an error. The intersection of the corrupted row and corrupted column pinpoints the exact bit that flipped, enabling **automatic error correction** without retransmission!

PART 1 (Continued): ADVANCED ERROR DETECTION METHODS

3. Checksum Calculation — TB Pages 58–59

A checksum is a mathematical value calculated from a block of data before transmission.

1. The sender computes a numerical sum of all bytes in the data block using a predetermined algorithm.
2. If the sum exceeds 255 (1 byte), modulo 256 is applied ($X = \text{Sum } \% \text{ } 256$) and subtracted ($255 - X$) to determine the checksum value.
3. The checksum is appended to the block and transmitted.
4. Upon arrival, the recipient re-calculates the checksum on the received payload and compares it against the received checksum byte.
5. If the values match, transmission is accepted; if they disagree, an error occurred and a re-transmission request is issued.

4. Echo Checks — TB Pages 59–60

In an **Echo Check**, when data is sent to the recipient, the recipient immediately sends a copy of the received data back (echoes it) to the sender. The sender compares the echoed data with the original master copy.

- **Advantage:** Very simple logic; requires minimal processing power.
- **Critical Limitations:**
  - It doubles network bandwidth consumption (every packet is transmitted twice).
  - If a mismatch occurs, the sender cannot tell whether the error occurred during the original transmission to the recipient or during the return echo journey.

5. Check Digits (ISBN-13 & Modulo 11) — TB Pages 60–61

While parity and checksums detect *data transmission* errors, **Check Digits** are designed to detect human data entry errors (e.g. entering a barcode or account number).

Common human entry errors detected by Check Digits:

- **Transcription Error:** Typing an incorrect digit (e.g. typing `5389` instead of `5379`).
- **Transposition Error:** Swapping two adjacent digits (e.g. typing `5398` instead of `5389`).
- **Omission / Extra Digit Error:** Missing a digit or entering an extra digit.
- **Phonetic Error:** Mishearing numbers (e.g. typing `13` instead of `30`).

> **ISBN-13 Algorithm:**
> `Multiply alternating digits by 1 and 3:`
> `Sum = (d1 × 1) + (d2 × 3) + (d3 × 1) + (d4 × 3) + ... + (d12 × 3)`
> `Remainder = Sum % 10`
> `Check Digit = (Remainder == 0) ? 0 : (10 - Remainder)`

PART 1 (Continued): ARQ PROTOCOL & SUMMARY COMPARISON

6. Automatic Repeat Requests (ARQ) — TB Pages 61–62

ARQ is a network protocol that uses positive acknowledgements (ACK), negative acknowledgements (NAK), and timeout limits to achieve reliable data transmission over unreliable channels.

- **Positive Acknowledgement (ACK):** The receiver sends an ACK message upon receiving a clean packet with valid checksum/parity.
- **Negative Acknowledgement (NAK):** The receiver sends a NAK message when packet corruption is detected, prompting immediate resend.
- **Timeout Timer:** The sender starts a countdown timer immediately upon sending a packet. If no ACK or NAK is received before the timer expires (due to a lost packet), the sender automatically re-transmits the identical data block.

Detection Method	How It Works	Primary Advantage	Major Limitation
Parity Check	Appends 1 parity bit to ensure even or odd number of 1s in byte.	Extremely fast; tiny overhead (1 bit per byte).	Cannot detect even-numbered bit flips in same byte.
Parity Block	2D matrix of row parity bits and a vertical parity byte.	Pinpoints exact bit coordinates to auto-correct errors.	Ineffective if multiple bits in the same row and column fail.
Checksum	Calculates mathematical sum of block bytes and appends value.	High reliability across large files and data blocks.	Requires computational processing at both ends.
Echo Check	Receiver transmits copy of received data back to sender.	Simple implementation without complex math.	Doubles network traffic; cannot localize error direction.
Check Digit	Appended final digit calculated from prior digits using weights.	Detects human input, transposition and transcription errors.	Only used for manual entry/barcodes, not network packets.
ARQ Protocol	Combines error detection with ACK, NAK, and timeout resends.	Guarantees 100% complete data delivery across noisy channels.	Introduces latency while waiting for timeouts or re-sends.

PART 2: CHAPTER 2.3 — DATA SECURITY & ENCRYPTION (TB Pages 63–74)

When data is transmitted across public networks (such as the Internet), unauthorized eavesdroppers can intercept packets. **Encryption** transforms confidential data into an unreadable scrambled form to safeguard privacy.

1. Core Terminology — TB Page 63

- **Plaintext:** The original, readable source data before encryption is applied (e.g., `"Meet at 10 AM"`).
- **Ciphertext:** The scrambled, unreadable output produced after running plaintext through an encryption algorithm using an encryption key.
- **Encryption Algorithm:** The mathematical formula used to scramble the plaintext.
- **Key:** A binary sequence or secret value required by the algorithm to encrypt and decrypt the message.

2. Symmetric Encryption — TB Pages 63–67

In **Symmetric Encryption**, the exact same single secret key is used to both *encrypt* the plaintext and *decrypt* the ciphertext.

```
[Plaintext] + [Shared Secret Key] → [Ciphertext]
[Ciphertext] + [Shared Secret Key] → [Plaintext]
```

- **Advantage:** Computationally fast and efficient; ideal for encrypting massive files, hard drives, and streaming video.
- **The Key Distribution Problem:** Both the sender and recipient must possess the identical secret key. How do they share this key initially across an insecure network? If a hacker intercepts the key in transit, all future messages can be easily decrypted!
- **Brute Force Attacks:** Attackers use high-speed automated supercomputers to test every possible key combination until the correct one is discovered. Longer keys (e.g. AES 256-bit) make brute-force attacks mathematically impossible within human lifespans.

3. Asymmetric Encryption — TB Pages 68–70

To permanently solve the Key Distribution Problem, **Asymmetric Encryption** utilizes a mathematically linked pair of different keys:

1. **Public Key:** Made freely available to anyone in the world (e.g. published on a website directory). Anyone can use it to encrypt a message intended for the owner.
2. **Private Key:** Kept strictly confidential and secret by the owner. It is NEVER shared or transmitted across any network.

```
Rule 1: What is encrypted with the PUBLIC key can ONLY be decrypted by the
matching PRIVATE key.
Rule 2: It is mathematically infeasible to deduce the private key from the public
key.
```

PART 2 (Continued): ASYMMETRIC WORKFLOW & DIGITAL CERTIFICATES

4. Asymmetric Message Exchange Scenario

Suppose **Alice** wants to send a confidential banking message to **Bob**:

1. Alice obtains Bob's public key from a public registry.
2. Alice encrypts her message using **Bob's Public Key**.
3. The ciphertext is transmitted across the open Internet. Even if an eavesdropper intercepts the transmission, they cannot read it because only Bob's private key can decrypt it.
4. Bob receives the ciphertext and decrypts it using his own **Bob's Private Key**.

5. Digital Signatures & Non-Repudiation — TB Pages 71–72

Asymmetric keys can also be used in reverse to authenticate the sender's identity and prove the document has not been tampered with:

- **Hashing:** The sender runs the message through a one-way mathematical hash function (e.g. SHA-256) to produce a fixed-length string called a *message digest*. If even a single character in the message changes, the hash changes completely.
- **Digital Signature Creation:** The sender encrypts the hash using their own **Private Key**. This encrypted hash is the *Digital Signature*.
- **Verification:** The recipient decrypts the signature using the sender's **Public Key** to reveal the original hash. They then compute their own hash of the received message. If both hashes match, two things are guaranteed:
  1. **Authentication & Non-Repudiation:** Only the sender could have created the signature because only they hold their private key.
  2. **Data Integrity:** The message was not modified in transit.

6. SSL/TLS & Digital Certificates — TB Pages 73–74

How does a user know a website's public key really belongs to them and not an imposter? Web browsers use **Digital Certificates** issued by trusted third-party **Certificate Authorities (CAs)** (such as DigiCert or Let's Encrypt).

A Digital Certificate contains:

- The organization / domain name (e.g., `kinderland.edu`).
- The website's Public Key.
- The Digital Signature of the Certificate Authority.
- The Validity period (Start and Expiry dates) and Serial number.

SSL/TLS Handshake: When connecting to a secure website (HTTPS), the browser uses asymmetric encryption to authenticate the server and securely exchange a temporary *session key*. Once established, high-speed symmetric encryption is used for the remainder of the session!

PART 3: SUMMARY COMPARISON TABLE & CAMBRIDGE EXAM DRILL

Feature	Symmetric Encryption	Asymmetric Encryption
Number of Keys	1 key (shared secret key).	2 keys (mathematically linked Public & Private pair).
Encryption / Decryption	Same key encrypts and decrypts.	Public key encrypts; Private key decrypts.
Processing Speed	Very fast; low computational overhead.	Significantly slower (up to 1,000× slower).
Key Distribution	High risk: secret key must be shared across network.	Zero risk: public key can be shared openly anywhere.
Primary Usage	Encrypting large files, disk drives, live data streams.	Exchanging session keys, Digital Signatures, SSL handshakes.

Cambridge Exam Golden Tips

Tip 1 (Parity): When asked why a parity check failed to catch an error, always write: *"An even number of bits (e.g. 2 bits) flipped, so the parity count remained valid."*

Tip 2 (ARQ): Remember that an ARQ retransmission can be triggered by TWO things: either the sender receives a **NAK**, OR the sender's **timeout timer expires** before an ACK is received.

Tip 3 (Keys): In asymmetric encryption, remember the golden rule: *You ALWAYS encrypt using the RECIPIENT'S public key, and the recipient decrypts using their OWN private key.*